LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 7: Mini Proyek Terpadu TabungKu (To-Do List + Manajemen Keuangan)

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	7

1. Latar Belakang

Banyak mahasiswa kesulitan menabung karena pengeluaran kecil tidak tercatat. Padahal setiap kegiatan sehari-hari (kuliah, kerja kelompok, nongkrong, kerja paruh waktu) hampir selalu berkaitan dengan uang: ada yang mengeluarkan biaya, ada yang menghasilkan pemasukan. Pada mini proyek ini dibangun TabungKu, yaitu aplikasi to-do list yang setiap kegiatannya dapat dicatat keuangannya, lalu diringkas menjadi laporan harian untuk melatih kebiasaan menabung.

2. Tujuan Pembelajaran

Setelah mengerjakan mini proyek ini, mahasiswa diharapkan mampu:

1. Menganalisis kebutuhan sederhana dan menerjemahkannya menjadi aturan bisnis, rancangan data, dan daftar layar.
2. Mengintegrasikan materi Pertemuan 1–6 dalam satu aplikasi utuh: widget dan layout, form dan validasi, state, SQLite, shared_preferences, tema, named routes, NavigationBar, dan animasi.
3. Menulis query agregasi (COUNT, SUM, GROUP BY) untuk membuat laporan.
4. Menyusun kode ke dalam struktur folder yang rapi (model, data, state, widget, halaman).
5. Menguji aplikasi dengan skenario uji, mendokumentasikan, dan mendemonstrasikannya.

3. Deskripsi Aplikasi

Pengguna mendaftarkan kegiatan pada suatu tanggal. Untuk setiap kegiatan pengguna menandai salah satu dari tiga jenis: Tidak melibatkan uang, Pemasukan (dengan nominal), atau Pengeluaran (dengan nominal dan kategori). Kegiatan dapat dicentang selesai. Aplikasi

menghitung laporan harian dan membandingkan tabungan hari itu dengan target menabung yang ditetapkan pengguna.

3.1 Aturan Bisnis

- Setiap kegiatan memiliki: nama, tanggal, status selesai, jenis keuangan, nominal, dan (khusus pengeluaran) kategori.
- Tabungan harian = total pemasukan dikurangi total pengeluaran dari semua kegiatan pada tanggal tersebut, baik yang sudah selesai maupun belum (keputusan desain).
- Nominal wajib berupa angka lebih dari 0 bila jenis keuangan adalah pemasukan atau pengeluaran.
- Kategori pengeluaran: Makan, Transport, Belajar, Hiburan, Lainnya.
- Target tabungan harian dapat diatur pengguna (bawaan Rp 20.000) dan tersimpan setelah aplikasi ditutup.
- Bila tabungan negatif, aplikasi menampilkan peringatan; bila mencapai target, aplikasi memberi apresiasi.

3.2 Contoh Skenario Satu Hari

Kegiatan	Jenis	Nominal	Kategori	Selesai
Kuliah pagi dan makan siang	Pengeluaran	Rp 15.000	Makan	Ya
Pulang naik angkot	Pengeluaran	Rp 10.000	Transport	Ya
Kerja freelance desain poster	Pemasukan	Rp 50.000	-	Belum
Belajar Flutter di perpustakaan	Tidak	-	-	Belum

Hasil Laporan Hari Itu	Nilai
Jumlah kegiatan (selesai/total)	2/4
Total pemasukan	Rp 50.000
Total pengeluaran	Rp 25.000 (Makan Rp 15.000, Transport Rp 10.000)
Tabungan hari itu	Rp 25.000 (melebihi target Rp 20.000, jadi target tercapai)

3.3 Fitur Wajib

Kode	Fitur	Diimplementasikan di
F1	Daftar kegiatan hari ini dengan centang selesai, ubah, dan hapus (dengan konfirmasi)	beranda_page.dart, kegiatan_tile.dart

Kode	Fitur	Diimplementasikan di
F2	Form tambah/ubah kegiatan: nama, tanggal (date picker), jenis keuangan, nominal, kategori; semua tervalidasi	form_kegiatan_page.dart
F3	Penyimpanan data di SQLite (tabel kegiatan)	db_helper.dart
F4	Ringkasan hari ini: jumlah kegiatan, pemasukan, pengeluaran, tabungan	ringkasan_card.dart, DbHelper.ringkasan()
F5	Halaman Laporan: pilih tanggal, ringkasan, pengeluaran per kategori, dan 7 hari terakhir	laporan_page.dart
F6	Target tabungan harian + indikator kemajuan beranimasi	ringkasan_card.dart (TweenAnimationBuilder)
F7	Pengaturan: mode gelap, warna tema, target; tersimpan	pengaturan_page.dart, pengaturan.dart
F8	Navigasi tiga tab (NavigationBar) dan halaman form lewat named route	shell_page.dart, main.dart

4. Rancangan

4.1 Paket yang Digunakan

- sqflite dan path: database SQLite lokal.
- shared_preferences: menyimpan tema, warna, dan target tabungan.

```
flutter pub add sqflite path shared_preferences
```

4.2 Struktur Folder

```
lib/
  main.dart                       tema + named routes
  models/kegiatan.dart            Kegiatan, Tipe, Ringkasan
  data/db_helper.dart             SQLite (CRUD + query laporan)
  state/pengaturan.dart           tema, warna, target
  utils/format.dart               format tanggal dan rupiah
  widgets/ringkasan_card.dart     kartu ringkasan + progress target
  widgets/kegiatan_tile.dart      satu baris kegiatan
  pages/shell_page.dart           NavigationBar 3 tab
  pages/beranda page.dart         kegiatan hari ini
  pages/form_kegiatan_page.dart   tambah/ubah kegiatan
  pages/laporan_page.dart         laporan
  pages/pengaturan_page.dart      pengaturan
```

Pemisahan ini membuat setiap berkas hanya memiliki satu tanggung jawab: model berisi bentuk data, data berisi akses database, state berisi pengaturan, utils berisi fungsi bantu, widgets berisi komponen yang dipakai ulang, dan pages berisi layar.

4.3 Rancangan Tabel kegiatan

Kolom	Tipe	Keterangan
id	INTEGER, PK, AUTOINCREMENT	Kunci utama
judul	TEXT, NOT NULL	Nama kegiatan
tanggal	TEXT, NOT NULL	Format yyyy-MM-dd agar mudah dibandingkan dan diurutkan
selesai	INTEGER, 0/1	SQLite tidak memiliki tipe boolean
tipe	TEXT	tanpa, pemasukan, atau pengeluaran
nominal	INTEGER	Rupiah bulat (tanpa desimal); 0 bila tanpa uang
kategori	TEXT	Diisi hanya untuk pengeluaran, selain itu '-'

4.4 Alur Data dan Halaman

Halaman	Isi Utama	Sumber Data
Hari Ini	Kartu ringkasan, daftar kegiatan hari ini, tombol tambah	DbHelper.pada(), DbHelper.ringkasan()
Form Kegiatan	Nama, tanggal, jenis keuangan, nominal, kategori	Menulis lewat DbHelper.tambah() / ubah()
Laporan	Pilih tanggal, ringkasan, per kategori, 7 hari terakhir	ringkasan(), pengeluaranPerKategori(), tujuhHari()
Pengaturan	Slider target, saklar gelap, pilihan warna	shared_preferences

Menyegarkan tampilan. Tiga tab menampilkan data yang sama, sehingga perubahan di satu tempat harus terlihat di tempat lain. Solusinya adalah variabel global ValueNotifier<int> versiData di db_helper.dart yang dinaikkan setiap kali ada tambah, ubah, atau hapus. Halaman yang menampilkan data mendengarkannya dengan addListener lalu memuat ulang Future-nya, dan melepasnya dengan removeListener di dispose().

5. Implementasi

Pengerjaan dilakukan bertahap: rancangan, model dan utilitas, DbHelper, halaman Hari Ini, form, laporan, lalu pengaturan dan navigasi. Berikut kode dan penjelasan fungsi setiap berkas.

5.1 Model (models/kegiatan.dart)

```
// lib/models/kegiatan.dart
enum Tipe { tanpa, pemasukan, pengeluaran }
```

```
class Kegiatan {
  final int? id;
  final String judul;
  final String tanggal;      // format yyyy-MM-dd
  final bool selesai;
  final Tipe tipe;
  final int nominal;
  final String kategori;

  const Kegiatan({
    this.id,
    required this.judul,
    required this.tanggal,
    this.selesai = false,
    this.tipe = Tipe.tanpa,
    this.nominal = 0,
    this.kategori = '-',
  });

  Map<String, Object?> toMap() => {
        'id': id,
        'judul': judul,
        'tanggal': tanggal,
        'selesai': selesai ? 1 : 0,
        'tipe': tipe.name,
        'nominal': nominal,
        'kategori': kategori,
      };

  factory Kegiatan.fromMap(Map<String, Object?> m) => Kegiatan(
        id: m['id'] as int,
        judul: m['judul'] as String,
        tanggal: m['tanggal'] as String,
        selesai: (m['selesai'] as int) == 1,
        tipe: Tipe.values.byName(m['tipe'] as String),
        nominal: m['nominal'] as int,
        kategori: m['kategori'] as String,
      );
}

class Ringkasan {
  final int jumlahKegiatan;
  final int selesai;
  final int pemasukan;
  final int pengeluaran;

  const Ringkasan({
    required this.jumlahKegiatan,
    required this.selesai,
    required this.pemasukan,
    required this.pengeluaran,
  });
```

```
  int get tabungan => pemasukan - pengeluaran;
}

class HariRingkas {
  final String tanggal;
  final int pemasukan;
  final int pengeluaran;
  const HariRingkas(this.tanggal, this.pemasukan, this.pengeluaran);
  int get tabungan => pemasukan - pengeluaran;
}
```

Bagian Kode	Penjelasan
enum Tipe { tanpa, pemasukan, pengeluaran }	Tipe enumerasi untuk tiga jenis keuangan. Lebih aman daripada String karena salah ketik terdeteksi saat kompilasi.
class Kegiatan	Model satu baris tabel kegiatan.
final int? id	Nullable: null saat kegiatan baru, diisi otomatis oleh SQLite.
this.selesai = false, this.tipe = Tipe.tanpa	Nilai bawaan parameter bernama; kegiatan baru otomatis belum selesai dan tanpa uang.
toMap()	Mengubah objek menjadi Map untuk database: selesai disimpan 0/1 (karena SQLite tidak punya boolean) dan tipe disimpan sebagai teks lewat tipe.name.
Kegiatan.fromMap(...)	Membalik proses: (m['selesai'] as int) == 1 menjadi bool, dan Tipe.values.byName(...) mengubah teks kembali menjadi enum.
class Ringkasan	Menampung angka laporan sehari: jumlah kegiatan, selesai, pemasukan, dan pengeluaran.
int get tabungan => pemasukan - pengeluaran	Getter terhitung untuk tabungan; tidak disimpan, selalu dihitung dari dua angka lain.
class HariRingkas	Ringkasan satu hari untuk daftar 7 hari terakhir (tanggal, pemasukan, pengeluaran, dan tabungan).

5.2 Utilitas Format (utils/format.dart)

```
// lib/utils/format.dart
String formatTanggal(DateTime d) {
  final bulan = d.month.toString().padLeft(2, '0');
  final hari = d.day.toString().padLeft(2, '0');
  return '${d.year}-$bulan-$hari';
}

String hariIni() => formatTanggal(DateTime.now());

String formatRupiah(int angka) {
  final s = angka.abs().toString();
```

```
  final buf = StringBuffer();
  for (int i = 0; i < s.length; i++) {
    if (i > 0 && (s.length - i) % 3 == 0) buf.write('.');
    buf.write(s[i]);
  }
  final tanda = angka < 0 ? '-' : '';
  return '${tanda}Rp $buf';
}
```

Bagian Kode	Penjelasan
formatTanggal(DateTime d)	Mengubah DateTime menjadi teks yyyy-MM-dd. padLeft(2, '0') menambah angka 0 di depan sehingga bulan 3 menjadi 03.
hariIni()	Tanggal hari ini dalam format yang sama dengan yang disimpan di database.
formatRupiah(int angka)	Memberi titik pemisah ribuan dengan menyisipkan "." pada kelipatan tiga digit dari kanan, misalnya 200000 menjadi Rp 200.000.
angka.abs()	Memakai nilai mutlak agar tanda minus tidak ikut diproses sebagai digit.
angka < 0 ? '-' : ''	Menaruh tanda minus di depan Rp untuk angka negatif, contoh -Rp 45.000.
StringBuffer	Penampung teks yang efisien untuk menyusun string sedikit demi sedikit.

5.3 Akses Data (data/db_helper.dart)

```
// lib/data/db_helper.dart
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import '../models/kegiatan.dart';
import '../utils/format.dart';

final versiData = ValueNotifier<int>(0);

class DbHelper {
  static Database? _db;

  static Future<Database> get _database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'tabungku.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
```

```
      return db.execute(
        'CREATE TABLE kegiatan('
        'id INTEGER PRIMARY KEY AUTOINCREMENT, '
        'judul TEXT NOT NULL, '
        'tanggal TEXT NOT NULL, '
        'selesai INTEGER NOT NULL, '
        'tipe TEXT NOT NULL, '
        'nominal INTEGER NOT NULL, '
        'kategori TEXT NOT NULL)',
      );
    },
  );
  return _db!;
}

static Future<void> tambah(Kegiatan k) async {
  final db = await _database;
  await db.insert('kegiatan', k.toMap());
  versiData.value++;
}

static Future<void> ubah(Kegiatan k) async {
  final db = await _database;
  await db.update('kegiatan', k.toMap(),
      where: 'id = ?', whereArgs: [k.id]);
  versiData.value++;
}

static Future<void> ubahSelesai(int id, bool selesai) async {
  final db = await _database;
  await db.update('kegiatan', {'selesai': selesai ? 1 : 0},
      where: 'id = ?', whereArgs: [id]);
  versiData.value++;
}

static Future<void> hapus(int id) async {
  final db = await _database;
  await db.delete('kegiatan', where: 'id = ?', whereArgs: [id]);
  versiData.value++;
}

static Future<List<Kegiatan>> pada(String tanggal) async {
  final db = await _database;
  final rows = await db.query('kegiatan',
      where: 'tanggal = ?', whereArgs: [tanggal],
      orderBy: 'id DESC');
  return rows.map(Kegiatan.fromMap).toList();
}

static Future<Ringkasan> ringkasan(String tanggal) async {
  final db = await _database;
  final hasil = await db.rawQuery('''
    SELECT COUNT(*) AS jumlah,
           COALESCE(SUM(selesai), 0) AS selesai,
```

```
            COALESCE(SUM(CASE WHEN tipe = 'pemasukan'
                        THEN nominal END), 0) AS masuk,
            COALESCE(SUM(CASE WHEN tipe = 'pengeluaran'
                        THEN nominal END), 0) AS keluar
      FROM kegiatan
      WHERE tanggal = ?
    ''', [tanggal]);
    final baris = hasil.first;
    return Ringkasan(
      jumlahKegiatan: baris['jumlah'] as int,
      selesai: baris['selesai'] as int,
      pemasukan: baris['masuk'] as int,
      pengeluaran: baris['keluar'] as int,
    );
  }

  static Future<Map<String, int>> pengeluaranPerKategori(
      String tanggal) async {
    final db = await _database;
    final rows = await db.rawQuery('''
      SELECT kategori, SUM(nominal) AS total
      FROM kegiatan
      WHERE tanggal = ? AND tipe = 'pengeluaran'
      GROUP BY kategori
      ORDER BY total DESC
    ''', [tanggal]);
    return {
      for (final r in rows) r['kategori'] as String: r['total'] as int
    };
  }

  static Future<List<HariRingkas>> tujuhHari(String akhir) async {
    final db = await _database;
    final mulai =
        DateTime.parse(akhir).subtract(const Duration(days: 6));
    final rows = await db.rawQuery('''
      SELECT tanggal,
            COALESCE(SUM(CASE WHEN tipe = 'pemasukan'
                        THEN nominal END), 0) AS masuk,
            COALESCE(SUM(CASE WHEN tipe = 'pengeluaran'
                        THEN nominal END), 0) AS keluar
      FROM kegiatan
      WHERE tanggal BETWEEN ? AND ?
      GROUP BY tanggal
      ORDER BY tanggal DESC
    ''', [formatTanggal(mulai), akhir]);
    return rows
        .map((r) => HariRingkas(r['tanggal'] as String,
            r['masuk'] as int, r['keluar'] as int))
        .toList();
  }
}
```

Bagian Kode	Penjelasan
final versiData = ValueNotifier<int>(0)	Penanda perubahan data yang bersifat global. Angkanya dinaikkan setiap data berubah, dan halaman yang mendengarkannya memuat ulang tampilan.
static Future<Database> get _database	Getter pribadi (diawali _) yang membuka database sekali saja lalu menyimpannya di _db.
onCreate: CREATE TABLE kegiatan(...)	Membuat tabel saat database pertama kali dibuat, sesuai rancangan kolom.
tambah(Kegiatan k)	Create: db.insert lalu versiData.value++ agar halaman lain tahu data berubah.
ubah(Kegiatan k)	Update: db.update dengan where 'id = ?' dan whereArgs sehingga aman dari SQL injection.
ubahSelesai(int id, bool selesai)	Update sebagian: hanya mengubah kolom selesai tanpa menyentuh kolom lain, dipakai oleh checkbox.
hapus(int id)	Delete: menghapus baris dengan id tertentu lalu menaikkan versiData.
pada(String tanggal)	Read: mengambil semua kegiatan pada satu tanggal, diurutkan dari id terbesar (terbaru di atas).
ringkasan(String tanggal)	Satu query agregasi yang menghasilkan semua angka ringkasan. COUNT(*) menghitung jumlah kegiatan; SUM(selesai) menghitung yang selesai karena nilainya 0/1.
CASE WHEN tipe = 'pemasukan' THEN nominal END	Memilah: nominal hanya ikut dijumlahkan bila tipenya sesuai (selain itu NULL dan diabaikan SUM).
COALESCE(..., 0)	Mengganti NULL dengan 0 saat tidak ada data, sehingga tidak muncul galat Null is not a subtype of int.
hasil.first	Query agregasi selalu menghasilkan tepat satu baris, jadi cukup diambil baris pertama.
pengeluaranPerKategori(...)	GROUP BY kategori menjumlahkan pengeluaran per kategori; ORDER BY total DESC mengurutkan dari terbesar. Hasilnya diubah menjadi Map kategori ke total.
tujuhHari(String akhir)	Mengambil total masuk dan keluar per tanggal untuk 7 hari (BETWEEN ? AND ?, GROUP BY tanggal). Tanggal mulai dihitung dengan subtract(Duration(days: 6)).
{ for (final r in rows) ...: ... }	Collection for: membentuk Map dari hasil query dalam satu ekspresi.

5.4 Pengaturan (state/pengaturan.dart)

```
// lib/state/pengaturan.dart
```

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class Pengaturan {
  static const pilihanWarna = [
    Colors.indigo, Colors.teal, Colors.deepOrange, Colors.pink,
  ];

  static final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
  static final warnaIndex = ValueNotifier<int>(0);
  static final target = ValueNotifier<int>(20000);

  static Color get warna => pilihanWarna[warnaIndex.value];

  static Future<void> muat() async {
    final prefs = await SharedPreferences.getInstance();
    themeMode.value = (prefs.getBool('gelap') ?? false)
        ? ThemeMode.dark : ThemeMode.light;
    warnaIndex.value = prefs.getInt('warna') ?? 0;
    target.value = prefs.getInt('target') ?? 20000;
  }

  static Future<void> setGelap(bool v) async {
    themeMode.value = v ? ThemeMode.dark : ThemeMode.light;
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', v);
  }

  static Future<void> setWarna(int i) async {
    warnaIndex.value = i;
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt('warna', i);
  }

  static Future<void> setTarget(int v) async {
    target.value = v;
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt('target', v);
  }
}
```

Bagian Kode	Penjelasan
pilihanWarna	Daftar empat warna dasar tema: indigo, teal, deepOrange, pink.
themeMode, warnaIndex, target	Tiga ValueNotifier yang menyimpan pengaturan aktif. Widget yang mendengarkannya otomatis diperbarui saat nilainya berubah.
static Color get warna	Getter yang mengambil warna dari pilihanWarna sesuai indeks aktif. Yang disimpan adalah indeksnya, bukan warnanya, agar sederhana.

Bagian Kode	Penjelasan
muat()	Dipanggil sekali sebelum aplikasi berjalan: membaca nilai tersimpan, dengan ?? sebagai nilai bawaan (terang, warna pertama, target 20.000).
setGelap / setWarna / setTarget	Mengubah nilai di ValueNotifier agar tampilan langsung berubah, lalu menyimpannya ke shared_preferences agar tetap ada setelah aplikasi ditutup.

5.5 Titik Masuk dan Routing (main.dart)

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'models/kegiatan.dart';
import 'pages/form_kegiatan_page.dart';
import 'pages/shell_page.dart';
import 'state/pengaturan.dart';

ThemeData buatTema(Color seed, Brightness brightness) {
  return ThemeData(
    useMaterial3: true,
    colorSchemeSeed: seed,
    brightness: brightness,
  );
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Pengaturan.muat();
  runApp(const TabungKuApp());
}

class TabungKuApp extends StatelessWidget {
  const TabungKuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: Listenable.merge(
          [Pengaturan.themeMode, Pengaturan.warnaIndex]),
      builder: (context, _) {
        return MaterialApp(
          title: 'TabungKu',
          debugShowCheckedModeBanner: false,
          theme: buatTema(Pengaturan.warna, Brightness.light),
          darkTheme: buatTema(Pengaturan.warna, Brightness.dark),
          themeMode: Pengaturan.themeMode.value,
          initialRoute: '/',
          routes: {
            '/': (_) => const ShellPage(),
          },
```

```
          onGenerateRoute: (settings) {
            if (settings.name == '/form') {
              final kegiatan = settings.arguments as Kegiatan?;
              return MaterialPageRoute(
                builder: (_) =>
                    FormKegiatanPage(kegiatan: kegiatan),
                settings: settings,
              );
            }
            return null;
          },
        );
      },
    );
  }
}
```

Bagian Kode	Penjelasan
buatTema(seed, brightness)	Membuat tema Material 3 dari satu warna dasar dan kecerahan (terang atau gelap).
WidgetsFlutterBinding.ensureInitialized()	Wajib dipanggil bila ada proses await sebelum runApp, agar plugin seperti shared_preferences siap dipakai.
await Pengaturan.muat()	Memuat tema dan target tersimpan sebelum tampilan pertama muncul, sehingga tidak ada kedipan tema.
ListenableBuilder + Listenable.merge	Membangun ulang MaterialApp saat mode atau warna berubah, sehingga seluruh aplikasi berganti tema.
theme, darkTheme, themeMode	Tema terang, tema gelap, dan pilihan yang dipakai.
routes: {'/': ShellPage}	Rute bernama untuk halaman utama.
onGenerateRoute	Menangani rute '/form' yang membawa argumen. settings.arguments as Kegiatan? bernilai null untuk tambah dan terisi untuk ubah.

5.6 Navigasi Tab (pages/shell_page.dart)

```
// lib/pages/shell_page.dart
class ShellPage extends StatefulWidget {
  const ShellPage({super.key});

  @override
  State<ShellPage> createState() => _ShellPageState();
}

class _ShellPageState extends State<ShellPage> {
  int _index = 0;

  static const _halaman = [
```

```
    BerandaPage(), LaporanPage(), PengaturanPage(),
  ];
  static const _judul = [
    'TabungKu - Hari Ini', 'Laporan', 'Pengaturan',
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_judul[_index])),
      body: IndexedStack(index: _index, children: _halaman),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _index,
        onDestinationSelected: (i) => setState(() => _index = i),
        destinations: const [
          NavigationDestination(
              icon: Icon(Icons.today), label: 'Hari Ini'),
          NavigationDestination(
              icon: Icon(Icons.bar_chart), label: 'Laporan'),
          NavigationDestination(
              icon: Icon(Icons.settings), label: 'Pengaturan'),
        ],
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
_index	Indeks tab yang sedang aktif.
IndexedStack	Menampilkan satu tab namun menjaga ketiganya tetap hidup, sehingga state dan listener tiap tab tidak hilang saat berpindah.
NavigationBar + NavigationDestination	Bilah navigasi bawah dengan tiga tujuan: Hari Ini, Laporan, Pengaturan.
onDestinationSelected	Mengubah _index lewat setState.
_judul[_index]	Judul AppBar menyesuaikan tab aktif.

5.7 Komponen: Kartu Ringkasan (widgets/ringkasan_card.dart)

```
// lib/widgets/ringkasan card.dart
class RingkasanCard extends StatelessWidget {
  final Ringkasan ringkasan;
  final int target;
  const RingkasanCard(
      {super.key, required this.ringkasan, required this.target});

  Widget _kolom(String label, int nilai, Color warna) {
    return Column(
```

```
    children: [
      Text(label),
      Text(formatRupiah(nilai),
          style: TextStyle(
              fontWeight: FontWeight.bold, color: warna)),
    ],
  );
}

@override
Widget build(BuildContext context) {
  final cs = Theme.of(context).colorScheme;
  final tabungan = ringkasan.tabungan;
  final progres =
      target <= 0 ? 0.0 : (tabungan / target).clamp(0.0, 1.0);

  return Card(
    margin: const EdgeInsets.all(16),
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text('Selesai: ${ringkasan.selesai}/'
                  '${ringkasan.jumlahKegiatan}'),
              Text('Target: ${formatRupiah(target)}'),
            ],
          ),
          const SizedBox(height: 8),
          TweenAnimationBuilder<double>(
            tween: Tween<double>(begin: 0, end: progres),
            duration: const Duration(milliseconds: 600),
            curve: Curves.easeOut,
            builder: (context, nilai, _) =>
                LinearProgressIndicator(value: nilai),
          ),
          const SizedBox(height: 12),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              _kolom('Masuk', ringkasan.pemasukan, Colors.green),
              _kolom('Keluar', ringkasan.pengeluaran, cs.error),
              _kolom('Tabungan', tabungan,
                  tabungan < 0 ? cs.error : cs.primary),
            ],
          ),
          const SizedBox(height: 8),
          if (tabungan < 0)
            Text(
              '\u26A0\uFE0F Peringatan: Tabungan kamu minus!',
              style: TextStyle(color: cs.error, fontSize: 12),
            )
```

```
            else if (target > 0 && tabungan >= target)
              const Text(
                '\u{1F389} Target menabung harian tercapai!',
                style: TextStyle(color: Colors.green, fontSize: 12),
              ),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
RingkasanCard(ringkasan, target)	Widget yang dipakai ulang di tab Hari Ini dan Laporan dengan data berbeda.
_kolom(label, nilai, warna)	Method pembantu yang membuat satu kolom (label dan angka berwarna) agar tidak menulis kode yang sama tiga kali.
(tabungan / target).clamp(0.0, 1.0)	Persentase kemajuan dibatasi antara 0 dan 1 agar progress bar tidak lebih dari penuh atau negatif.
target <= 0 ? 0.0 : ...	Mencegah pembagian dengan nol.
TweenAnimationBuilder<double>	Animasi implisit untuk F6: saat nilai akhir (end) berubah, bar bergerak halus selama 600 ms dengan kurva easeOut.
LinearProgressIndicator(value: nilai)	Bar kemajuan menuju target tabungan.
cs.error / cs.primary	Warna diambil dari tema (bukan ditulis manual) sehingga ikut berubah di mode gelap.
if (tabungan < 0) ... else if (...)	Pesan kondisional: peringatan merah bila minus, apresiasi hijau bila mencapai target, tidak ada pesan di antaranya.
Row(spaceBetween)	Meletakkan "Selesai" di kiri dan "Target" di kanan.
Row(spaceEvenly)	Membagi tiga kolom angka dengan jarak merata.

5.8 Komponen: Baris Kegiatan (widgets/kegiatan_tile.dart)

```
// lib/widgets/kegiatan_tile.dart
class KegiatanTile extends StatelessWidget {
  final Kegiatan kegiatan;
  final ValueChanged<bool?> onSelesai;
  final VoidCallback onUbah;
  final VoidCallback onHapus;

  const KegiatanTile({
    super.key,
    required this.kegiatan,
    required this.onSelesai,
```

```
    required this.onUbah,
    required this.onHapus,
  });

  @override
  Widget build(BuildContext context) {
    final k = kegiatan;
    final cs = Theme.of(context).colorScheme;
    final masuk = k.tipe == Tipe.pemasukan;
    final tanda = masuk ? '+' : '-';

    return ListTile(
      leading: Checkbox(value: k.selesai, onChanged: onSelesai),
      title: Text(
        k.judul,
        style: TextStyle(
          decoration:
              k.selesai ? TextDecoration.lineThrough : null,
        ),
      ),
      subtitle: Text('${k.tipe.name.toUpperCase()} - ${k.kategori}'),
      onTap: onUbah,
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          if (k.tipe != Tipe.tanpa)
            Text(
              '$tanda${formatRupiah(k.nominal)}',
              style: TextStyle(
                fontSize: 11,
                fontWeight: FontWeight.bold,
                color: masuk ? Colors.green : cs.error,
              ),
            ),
          IconButton(
              icon: const Icon(Icons.delete), onPressed: onHapus),
        ],
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
ValueChanged<bool?> onSelesai	Tipe callback yang menerima nilai checkbox baru.
VoidCallback onUbah / onHapus	Callback tanpa parameter; aksi sebenarnya ditentukan oleh halaman pemanggil sehingga widget ini murni tampilan.
Checkbox(value: k.selesai)	Centang selesai; onChanged meneruskan nilai ke halaman.
TextDecoration.lineThrough	Judul dicoret bila kegiatan selesai.

Bagian Kode	Penjelasan
k.tipe.name.toUpperCase()	Menampilkan jenis dengan huruf kapital, contoh PEMASUKAN.
if (k.tipe != Tipe.tanpa) Text(...)	Nominal hanya tampil bila ada uang, dengan tanda + hijau untuk pemasukan dan tanda - merah untuk pengeluaran.
Row(mainAxisSize: MainAxisSize.min)	Menaruh nominal dan ikon hapus berdampingan di sisi kanan dengan lebar seperlunya.

5.9 Halaman Hari Ini (pages/beranda_page.dart)

```
// lib/pages/beranda_page.dart
class DataHari {
  final Ringkasan ringkasan;
  final List<Kegiatan> daftar;
  const DataHari(this.ringkasan, this.daftar);
}

class _BerandaPageState extends State<BerandaPage> {
  late Future<DataHari> _future;

  Future<DataHari> _ambil() async {
    final tgl = hariIni();
    final ringkasan = await DbHelper.ringkasan(tgl);
    final daftar = await DbHelper.pada(tgl);
    return DataHari(ringkasan, daftar);
  }

  @override
  void initState() {
    super.initState();
    _future = _ambil();
    versiData.addListener(_muat);
  }

  @override
  void dispose() {
    versiData.removeListener(_muat);
    super.dispose();
  }

  void _muat() {
    setState(() {
      _future = _ambil();   // Future baru -> FutureBuilder memuat ulang
    });
  }

  Future<void> _konfirmasiHapus(Kegiatan k) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
```

```
      title: const Text('Hapus kegiatan?'),
      content: Text('"${k.judul}" akan dihapus.'),
      actions: [
        TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Batal')),
        TextButton(
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Hapus')),
      ],
    ),
  );
  if (yakin == true) await DbHelper.hapus(k.id!);
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: FutureBuilder<DataHari>(
      future: _future,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return Center(child: Text('Galat: ${snapshot.error}'));
        }
        final data = snapshot.data!;
        return Column(
          children: [
            ValueListenableBuilder<int>(
              valueListenable: Pengaturan.target,
              builder: (_, target, __) => RingkasanCard(
                  ringkasan: data.ringkasan, target: target),
            ),
            Expanded(
              child: data.daftar.isEmpty
                  ? const Center(
                      child: Text('Belum ada kegiatan hari ini.'))
                  : ListView.builder(
                      itemCount: data.daftar.length,
                      itemBuilder: (context, i) {
                        final k = data.daftar[i];
                        return KegiatanTile(
                          kegiatan: k,
                          onSelesai: (v) => DbHelper
                              .ubahSelesai(k.id!, v ?? false),
                          onUbah: () => Navigator.pushNamed(
                              context, '/form', arguments: k),
                          onHapus: () => _konfirmasiHapus(k),
                        );
                      },
                    ),
            ),
```

```
            ],
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.pushNamed(context, '/form'),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class DataHari	Membungkus ringkasan dan daftar kegiatan agar satu FutureBuilder dapat memuat keduanya sekaligus.
_ambil()	Mengambil ringkasan dan daftar untuk tanggal hari ini dari database.
initState: _future = _ambil(); addListener(_muat)	Memuat data pertama kali dan mendaftarkan _muat sebagai pendengar versiData.
dispose: removeListener(_muat)	Melepas pendengar saat halaman dibuang. Bila lupa, listener tetap memanggil setState pada halaman yang sudah tidak ada dan menimbulkan galat dan kebocoran memori.
_muat()	Membuat Future baru lewat setState sehingga FutureBuilder memuat ulang saat data berubah.
_konfirmasiHapus(Kegiatan k)	Menampilkan AlertDialog Batal/Hapus. Data baru dihapus bila hasilnya true, sesuai skenario uji 7.
FutureBuilder<DataHari>	Menangani tiga status: loading, galat, dan data.
ValueListenableBuilder<int>(Pengaturan.target)	Kartu ringkasan ikut berubah segera ketika target diubah di tab Pengaturan.
data.daftar.isEmpty ? Text(...) : ListView.builder	Kondisi kosong: tampil "Belum ada kegiatan hari ini."
DbHelper.ubahSelesai(k.id!, v ?? false)	Mencentang kegiatan menyimpan status ke database, lalu versiData memicu pembaruan ringkasan.
Navigator.pushNamed(context, '/form', arguments: k)	Membuka form lewat named route; argumen k berisi kegiatan yang akan diubah, tanpa argumen berarti tambah baru.
FloatingActionButton	Tombol + untuk menambah kegiatan.

5.10 Form Kegiatan (pages/form_kegiatan_page.dart)

```
// lib/pages/form_kegiatan_page.dart
```

```
class _FormKegiatanPageState extends State<FormKegiatanPage> {
  static const _kategori = [
    'Makan', 'Transport', 'Belajar', 'Hiburan', 'Lainnya'
  ];

  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _nominal;
  late DateTime _tanggal;
  late Tipe _tipe;
  String? _kategoriPilih;

  @override
  void initState() {
    super.initState();
    final k = widget.kegiatan;
    _judul = TextEditingController(text: k?.judul ?? '');
    _nominal = TextEditingController(
        text: (k != null && k.nominal > 0) ? '${k.nominal}' : '');
    _tanggal = k == null ? DateTime.now() : DateTime.parse(k.tanggal);
    _tipe = k?.tipe ?? Tipe.tanpa;
    _kategoriPilih =
        (k != null && k.tipe == Tipe.pengeluaran) ? k.kategori : null;
  }

  @override
  void dispose() {
    _judul.dispose();
    _nominal.dispose();
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final hasil = await showDatePicker(
      context: context,
      initialDate: _tanggal,
      firstDate: DateTime(2020),
      lastDate: DateTime(2100),
    );
    if (hasil != null) setState(() => _tanggal = hasil);
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final adaUang = _tipe != Tipe.tanpa;
    final k = Kegiatan(
      id: widget.kegiatan?.id,
      judul: _judul.text.trim(),
      tanggal: formatTanggal(_tanggal),
      selesai: widget.kegiatan?.selesai ?? false,
      tipe: _tipe,
      nominal: adaUang ? int.parse(_nominal.text) : 0,
      kategori: _tipe == Tipe.pengeluaran ? _kategoriPilih! : '-',
    );
```

```
  if (widget.kegiatan == null) {
    await DbHelper.tambah(k);
  } else {
    await DbHelper.ubah(k);
  }
  if (!mounted) return;
  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.kegiatan == null;
  return Scaffold(
    appBar: AppBar(
        title: Text(baru ? 'Tambah Kegiatan' : 'Ubah Kegiatan')),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _judul,
            decoration:
                const InputDecoration(labelText: 'Nama Kegiatan'),
            validator: (v) => (v == null || v.trim().isEmpty)
                ? 'Nama kegiatan wajib diisi' : null,
          ),
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: Text('Tanggal: ${formatTanggal(_tanggal)}'),
            trailing: const Icon(Icons.calendar_today),
            onTap: _pilihTanggal,
          ),
          DropdownButtonFormField<Tipe>(
            value: _tipe,
            decoration:
                const InputDecoration(labelText: 'Jenis Keuangan'),
            items: Tipe.values
                .map((t) => DropdownMenuItem(
                    value: t, child: Text(t.name.toUpperCase())))
                .toList(),
            onChanged: (v) => setState(() => _tipe = v!),
          ),
          if (_tipe != Tipe.tanpa)
            TextFormField(
              controller: _nominal,
              keyboardType: TextInputType.number,
              decoration:
                  const InputDecoration(labelText: 'Nominal (Rp)'),
              validator: (v) {
                final n = int.tryParse(v ?? '');
                if (n == null || n <= 0) {
                  return 'Nominal harus angka lebih dari 0';
                }
```

```
                  return null;
                },
              ),
            if (_tipe == Tipe.pengeluaran)
              DropdownButtonFormField<String>(
                value: _kategoriPilih,
                decoration:
                    const InputDecoration(labelText: 'Kategori'),
                items: _kategori
                    .map((k) => DropdownMenuItem(
                        value: k, child: Text(k)))
                    .toList(),
                onChanged: (v) =>
                    setState(() => _kategoriPilih = v),
                validator: (v) =>
                    v == null ? 'Pilih kategori' : null,
              ),
            const SizedBox(height: 24),
            ElevatedButton(
                onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
final Kegiatan? kegiatan	Null berarti mode tambah, terisi berarti mode ubah (satu form untuk dua keperluan).
late DateTime _tanggal; late Tipe _tipe	State form yang diisi di initState dari data lama atau nilai bawaan (hari ini, Tipe.tanpa).
_pilihTanggal()	Membuka date picker (showDatePicker); bila pengguna memilih tanggal, state diperbarui.
_simpan()	Validasi dulu; bila lolos membuat objek Kegiatan, memilih tambah atau ubah, lalu menutup form.
nominal: adaUang ? int.parse(...) : 0	Bila jenis Tidak, nominal otomatis 0 dan kategori '-' sesuai aturan data.
kategori: _tipe == Tipe.pengeluaran ? _kategoriPilih! : '-'	Kategori hanya dipakai untuk pengeluaran.
DropdownButtonFormField<Tipe>	Pilihan jenis keuangan. Berganti pilihan memanggil setState sehingga kolom di bawahnya muncul atau hilang.
if (_tipe != Tipe.tanpa) TextFormField(...)	Collection if: kolom nominal hanya tampil bila ada uang.

Bagian Kode	Penjelasan
if (_tipe == Tipe.pengeluaran) Dropdown(...)	Kolom kategori hanya tampil untuk pengeluaran.
validator nominal	int.tryParse mengubah teks menjadi angka (null bila bukan angka); ditolak bila null atau tidak lebih dari 0.
validator judul	Nama wajib diisi; spasi dibuang dengan trim().
dispose()	Melepas kedua TextEditingController.

5.11 Halaman Laporan (pages/laporan_page.dart)

```
// lib/pages/laporan_page.dart
class LaporanData {
  final Ringkasan ringkasan;
  final Map<String, int> perKategori;
  final List<HariRingkas> tujuhHari;
  const LaporanData(this.ringkasan, this.perKategori, this.tujuhHari);
}

class _LaporanPageState extends State<LaporanPage> {
  DateTime _tanggal = DateTime.now();
  late Future<LaporanData> _future;

  Future<LaporanData> _ambil() async {
    final tgl = formatTanggal(_tanggal);
    return LaporanData(
      await DbHelper.ringkasan(tgl),
      await DbHelper.pengeluaranPerKategori(tgl),
      await DbHelper.tujuhHari(tgl),
    );
  }

  @override
  void initState() {
    super.initState();
    _future = _ambil();
    versiData.addListener(_muat);
  }

  @override
  void dispose() {
    versiData.removeListener(_muat);
    super.dispose();
  }

  void _muat() => setState(() => _future = _ambil());

  Future<void> _pilihTanggal() async {
    final hasil = await showDatePicker(
      context: context,
```

```
    initialDate: _tanggal,
    firstDate: DateTime(2020),
    lastDate: DateTime(2100),
  );
  if (hasil != null) {
    _tanggal = hasil;
    _muat();
  }
}

@override
Widget build(BuildContext context) {
  return FutureBuilder<LaporanData>(
    future: _future,
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: CircularProgressIndicator());
      }
      if (snapshot.hasError) {
        return Center(child: Text('Galat: ${snapshot.error}'));
      }
      final d = snapshot.data!;
      final tema = Theme.of(context);
      return ListView(
        children: [
          ListTile(
            title: Text('Pilih Tanggal: ${formatTanggal(_tanggal)}'),
            trailing: const Icon(Icons.calendar_month),
            onTap: _pilihTanggal,
          ),
          ValueListenableBuilder<int>(
            valueListenable: Pengaturan.target,
            builder: (_, target, __) => RingkasanCard(
                ringkasan: d.ringkasan, target: target),
          ),
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Text('Pengeluaran per kategori',
                style: tema.textTheme.titleMedium),
          ),
          if (d.perKategori.isEmpty)
            const ListTile(title: Text('Tidak ada pengeluaran.'))
          else
            for (final e in d.perKategori.entries)
              ListTile(
                dense: true,
                title: Text(e.key),
                trailing: Text(formatRupiah(e.value)),
              ),
          const Divider(),
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Text('7 hari terakhir',
                style: tema.textTheme.titleMedium),
```

```
            ),
            for (final h in d.tujuhHari)
              ListTile(
                dense: true,
                title: Text(h.tanggal),
                subtitle: Text('Masuk ${formatRupiah(h.pemasukan)}'
                    ' | Keluar ${formatRupiah(h.pengeluaran)}'),
                trailing: Text(
                  formatRupiah(h.tabungan),
                  style: TextStyle(
                    fontWeight: FontWeight.bold,
                    color: h.tabungan < 0
                        ? tema.colorScheme.error : null,
                  ),
                ),
              ),
          ],
        );
      },
    );
  }
}
```

Bagian Kode	Penjelasan
class LaporanData	Membungkus tiga hasil query (ringkasan, per kategori, 7 hari) dalam satu objek.
_ambil()	Memanggil ketiga query untuk tanggal yang dipilih dengan await berurutan.
_pilihTanggal()	Memilih tanggal laporan lewat date picker, lalu memuat ulang laporan.
versiData.addListener(_muat)	Laporan ikut berubah saat kegiatan ditambah, diubah, atau dihapus di tab lain tanpa restart (skenario uji 6).
for (final e in d.perKategori.entries)	Collection for di dalam children: membuat satu ListTile per kategori.
d.perKategori.isEmpty ? ... : ...	Kondisi kosong: "Tidak ada pengeluaran."
for (final h in d.tujuhHari)	Satu baris per hari berisi total masuk dan keluar; tabungan negatif berwarna merah dari tema.
tema.textTheme.titleMedium	Gaya judul bagian diambil dari tema.

5.12 Halaman Pengaturan (pages/pengaturan_page.dart)

```
// lib/pages/pengaturan_page.dart
class PengaturanPage extends StatelessWidget {
  const PengaturanPage({super.key});
```

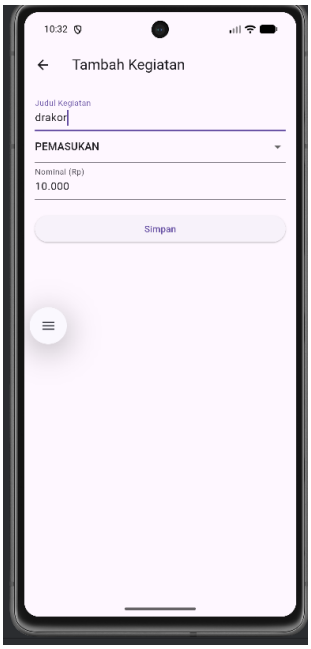
```
@override
Widget build(BuildContext context) {
  final tema = Theme.of(context);
  return ListenableBuilder(
    listenable: Listenable.merge([
      Pengaturan.themeMode,
      Pengaturan.warnaIndex,
      Pengaturan.target,
    ]),
    builder: (context, _) {
      return ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(
            'Target tabungan harian: '
            '${formatRupiah(Pengaturan.target.value)}',
            style: tema.textTheme.titleMedium,
          ),
          Slider(
            min: 10000,
            max: 200000,
            divisions: 19,
            label: formatRupiah(Pengaturan.target.value),
            value: Pengaturan.target.value.toDouble(),
            onChanged: (v) =>
                Pengaturan.target.value = v.round(),
            onChangeEnd: (v) => Pengaturan.setTarget(v.round()),
          ),
          SwitchListTile(
            title: const Text('Mode gelap'),
            value: Pengaturan.themeMode.value == ThemeMode.dark,
            onChanged: Pengaturan.setGelap,
          ),
          const SizedBox(height: 8),
          Text('Warna tema', style: tema.textTheme.titleMedium),
          const SizedBox(height: 8),
          Wrap(
            spacing: 12,
            children: [
              for (int i = 0;
                  i < Pengaturan.pilihanWarna.length; i++)
                GestureDetector(
                  onTap: () => Pengaturan.setWarna(i),
                  child: CircleAvatar(
                    backgroundColor: Pengaturan.pilihanWarna[i],
                    child: Pengaturan.warnaIndex.value == i
                        ? const Icon(Icons.check,
                            color: Colors.white)
                        : null,
                  ),
                ),
            ],
          ),
        ],
```

```
          );
        },
      );
    }
}
```

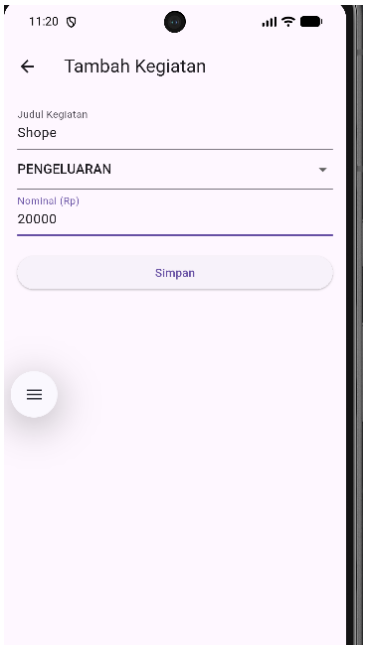
Bagian Kode	Penjelasan
ListenableBuilder + Listenable.merge	Membangun ulang halaman saat salah satu dari tiga pengaturan berubah.
Slider(min, max, divisions, value)	Penggeser target tabungan dari Rp 10.000 sampai Rp 200.000 dengan 19 langkah (kelipatan Rp 10.000).
label: formatRupiah(...)	Gelembung nilai muncul saat slider digeser.
onChanged / onChangeEnd	onChanged memperbarui nilai selama digeser agar tampilan langsung berubah; onChangeEnd menyimpan ke penyimpanan hanya saat jari dilepas, sehingga tidak menulis berkali-kali.
SwitchListTile(onChanged: Pengaturan.setGelap)	Saklar mode gelap; fungsi setGelap langsung dipakai sebagai callback.
Wrap + CircleAvatar + GestureDetector	Pilihan warna berupa lingkaran yang dapat diketuk; ikon centang menandai warna aktif.

6. Hasil Praktikum

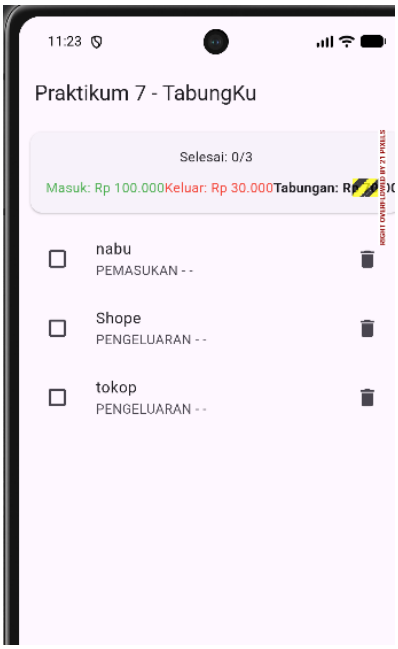
Hasil tahap awal pengerjaan (form, penyimpanan SQLite, dan daftar kegiatan) ditampilkan berikut.



Gambar 1. Form Tambah Kegiatan jenis Pemasukan



Gambar 2. Form Tambah Kegiatan jenis Pengeluaran



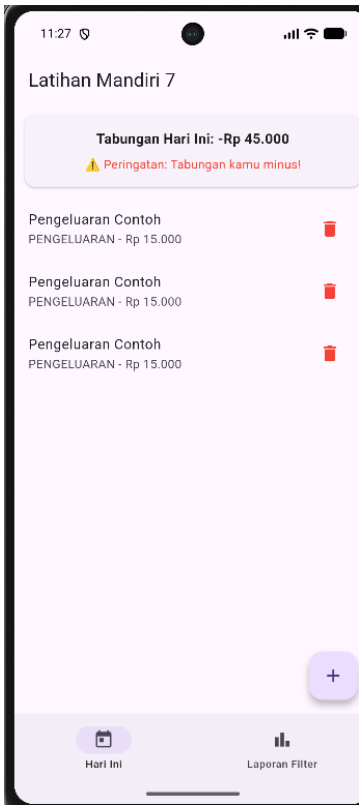
Gambar 3. Daftar kegiatan dan ringkasan awal

Gambar 1 menampilkan form dengan kegiatan "drakor" berjenis PEMASUKAN dan nominal Rp 10.000; kolom nominal muncul karena jenisnya bukan Tidak. Gambar 2 menampilkan kegiatan "Shope" berjenis PENGELUARAN dengan nominal 20000. Gambar 3 memperlihatkan halaman Hari Ini yang sudah memuat tiga kegiatan dari SQLite (nabu, Shope, tokop) dengan kartu ringkasan: Selesai 0/3, Masuk Rp 100.000, Keluar Rp 30.000.

Temuan: pada Gambar 3 muncul peringatan RIGHT OVERFLOWED BY 21 PIXELS pada baris Masuk/Keluar/Tabungan di kartu ringkasan, karena tiga teks diletakkan dalam satu Row tanpa pembagian ruang, sehingga teks Tabungan terpotong. Masalah ini sudah tidak muncul pada versi akhir, ketika ringkasan disusun sebagai tiga kolom Column dalam Row(spaceEvenly) dengan teks nominal lebih kecil, sehingga angka tampil utuh (lihat Gambar 6–8).

7. Latihan Mandiri

Latihan mandiri dikerjakan pada versi ringkas aplikasi bernama "Latihan Mandiri 7" yang memiliki dua tab, yaitu Hari Ini dan Laporan Filter. Latihan ini menguji dua hal: peringatan ketika tabungan negatif (skenario uji 10), dan laporan yang dapat difilter berdasarkan tanggal pilihan (fitur F5 dan skenario uji 8).



Gambar 4. Tab Hari Ini: tabungan negatif dan peringatan



Gambar 5. Tab Laporan Filter dengan pemilih tanggal

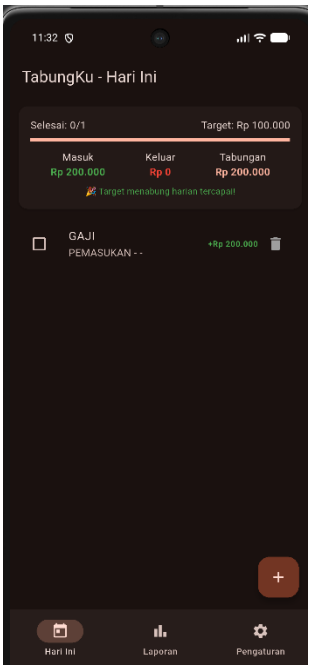
Pada Gambar 4 terdapat tiga kegiatan "Pengeluaran Contoh" masing-masing Rp 15.000, sehingga tabungan hari ini -Rp 45.000. Aplikasi menampilkan judul "Tabungan Hari Ini: -Rp 45.000" dan pesan peringatan berwarna merah "Tabungan kamu minus!", sesuai aturan bisnis. Pada Gambar 5 tab Laporan Filter menampilkan "Pilih Tanggal: 2026-10-24" dengan ikon kalender; daftar di bawahnya memuat kegiatan pada tanggal yang dipilih.

8. Tugas: Hasil Akhir Aplikasi TabungKu

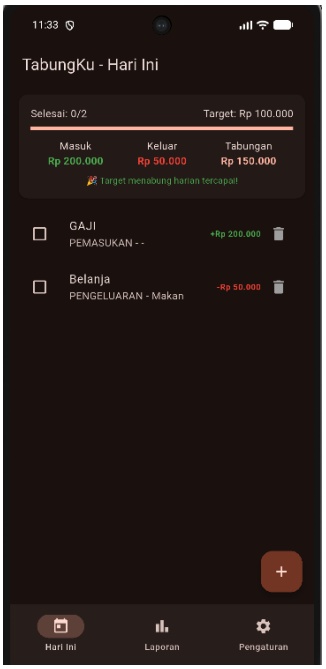
Hasil akhir adalah aplikasi TabungKu lengkap dengan tiga tab (Hari Ini, Laporan, Pengaturan), form dengan date picker, kartu ringkasan beranimasi dengan target tabungan, dan tema yang tersimpan. Gambar berikut diambil pada mode gelap dengan target tabungan Rp 100.000 yang diatur lewat tab Pengaturan.



Gambar 6. Form Tambah Kegiatan (mode gelap)



Gambar 7. Hari Ini: target tercapai



Gambar 8. Hari Ini: pemasukan dan pengeluaran

Gambar 6 menampilkan form dengan kolom Nama Kegiatan "GAJI", Tanggal 2026-10-24 yang dipilih lewat date picker, Jenis Keuangan PEMASUKAN, dan Nominal 200000. Gambar 7 menunjukkan hasilnya di tab Hari Ini: kartu ringkasan menampilkan Selesai 0/1, Target Rp 100.000, Masuk Rp 200.000, Keluar Rp 0, Tabungan Rp 200.000, bar kemajuan penuh, dan pesan hijau "Target menabung harian tercapai!"; kegiatan GAJI tampil dengan nominal +Rp 200.000 berwarna hijau. Gambar 8 menambahkan kegiatan "Belanja" berjenis PENGELUARAN kategori Makan sebesar -Rp 50.000, sehingga Keluar menjadi Rp 50.000 dan Tabungan turun menjadi Rp 150.000 namun tetap di atas target.

9. Skenario Uji

Hasil pengujian setiap skenario pada aplikasi. Kolom Hasil diisi berdasarkan bukti tangkapan layar yang tersedia; baris bertanda kuning perlu diisi setelah pengujiannya dijalankan.

No	Langkah	Hasil yang Diharapkan	Hasil
1	Tambah kegiatan "Belajar" jenis Tidak	Muncul di daftar; jumlah kegiatan naik; pemasukan/pengeluaran tidak berubah	Lulus
2	Tambah kegiatan jenis Pemasukan Rp 50.000	Pemasukan Rp 50.000; tabungan naik; nominal hijau dengan tanda +	Lulus. Nominal hijau +Rp 200.000 dan tabungan naik (Gambar 7).

No	Langkah	Hasil yang Diharapkan	Hasil
3	Tambah kegiatan jenis Pengeluaran Rp 15.000 kategori Makan	Pengeluaran naik; tabungan turun; kategori Makan muncul di Laporan	Lulus di Hari Ini: pengeluaran Makan naik dan tabungan turun Tampil di Laporan: perlu diperiksa.
4	Simpan kegiatan bernama kosong / nominal kosong / nominal 0	Pesan kesalahan tampil dan data tidak tersimpan	Lulus
5	Centang selesai dua kegiatan	Kartu ringkasan menunjukkan selesai/total yang benar; judul tercoret	Lulus
6	Ubah nominal kegiatan yang sudah ada	Ringkasan dan laporan ikut berubah tanpa restart	Lulus
7	Hapus kegiatan (pilih Batal, lalu Hapus)	Batal tidak menghapus; Hapus menghapus dan ringkasan berkurang	Lulus
8	Tambah kegiatan bertanggal kemarin	Tidak muncul di Hari Ini; muncul di Laporan pada tanggal itu dan pada daftar 7 hari	Lulus
9	Ubah target dan warna, aktifkan mode gelap, tutup aplikasi lalu buka lagi	Semua pengaturan dan data tetap	[isi: Lulus / Tidak setelah restart. Mode gelap dan target Rp 100.000 sudah terlihat pada Gambar 6–8]
10	Buat pengeluaran lebih besar dari pemasukan	Tabungan negatif berwarna merah dan muncul pesan peringatan	Lulus pada versi latihan: -Rp 45.000 dan peringatan merah (Gambar 4).

10. Evaluasi dan Perbaikan

- **Overflow kartu ringkasan.** Pada Gambar 3 baris Masuk/Keluar/Tabungan melebihi lebar layar sebesar 21 piksel. Perbaikan: membagi ruang dengan Row(spaceEvenly) berisi tiga Column, atau membungkus tiap kolom dengan Expanded, serta memakai ukuran teks lebih kecil.
- **Teks "PEMASUKAN - -".** Subtitle kegiatan menampilkan kategori '-' untuk kegiatan non-pengeluaran sehingga terbaca "PEMASUKAN - -" (Gambar 7). Perbaikan: tampilkan kategori hanya bila tipe pengeluaran.
- **Tabungan dihitung dari semua kegiatan.** Sesuai keputusan desain, kegiatan yang belum selesai ikut dihitung. Dampaknya dibahas pada jawaban refleksi nomor 4.

11. Pengembangan Lanjutan

Modul mewajibkan minimal dua fitur pengembangan lanjutan. Pilihan yang tersedia: rencana vs realisasi (hanya kegiatan selesai yang dihitung), laporan bulanan dengan grafik batang, pencarian dan filter, kategori kustom (onUpgrade), kutipan motivasi dari API publik, kegiatan berulang, dan ekspor laporan ke Clipboard.

12. Jawaban Pertanyaan Refleksi

1. Mengapa tanggal disimpan sebagai teks yyyy-MM-dd dan bukan format lain seperti dd/MM/yyyy?

SQLite tidak memiliki tipe tanggal, sehingga tanggal disimpan sebagai teks dan dibandingkan secara abjad. Pada format yyyy-MM-dd, urutan abjad sama dengan urutan kronologis karena bagian terbesar (tahun) berada di depan. Akibatnya perbandingan seperti WHERE tanggal = ?, BETWEEN ? AND ?, dan ORDER BY tanggal langsung benar tanpa konversi. Pada dd/MM/yyyy, teks diurutkan berdasarkan hari lebih dulu sehingga 01/12/2026 dianggap lebih kecil daripada 02/01/2027 dan hasil BETWEEN maupun urutan menjadi salah.

2. Mengapa laporan dihitung dengan query SUM di database, bukan dengan menjumlahkan di Dart? Kapan cara sebaliknya lebih tepat?

Query SUM dan GROUP BY dijalankan di dalam database sehingga hanya hasil akhir (satu baris angka) yang dikirim ke aplikasi, bukan ratusan baris yang harus dimuat ke memori lalu dijumlahkan. Ini lebih cepat, hemat memori, dan tetap efisien saat data bertambah banyak. Menjumlahkan di Dart lebih tepat bila data sudah ada di memori dan jumlahnya kecil (misalnya total dari daftar yang sudah ditampilkan), atau bila perhitungannya rumit dan bergantung pada logika aplikasi yang sulit ditulis dalam SQL. Pada Pertemuan 5 total pengeluaran dihitung di Dart karena daftarnya sudah dimuat.

3. Bagaimana tiga tab dapat menampilkan data yang selalu sinkron? Apa risikonya bila removeListener lupa dipanggil?

DbHelper menaikkan versiData setiap kali ada tambah, ubah, atau hapus. Setiap halaman yang menampilkan data mendaftarkan _muat sebagai pendengar versiData, sehingga ketika angkanya berubah mereka membuat Future baru dan FutureBuilder memuat ulang. IndexedStack menjaga ketiga tab tetap hidup agar pendengarnya tetap aktif. Bila removeListener lupa dipanggil di dispose(), pendengar tetap terdaftar walau halaman sudah dibuang. Saat versiData berubah, _muat dipanggil pada State yang sudah tidak aktif sehingga muncul galat setState() called after dispose(), dan objek halaman tidak bisa dibersihkan dari memori (kebocoran memori).

4. Apa dampak keputusan "semua kegiatan dihitung, bukan hanya yang selesai" terhadap kebiasaan menabung pengguna?

Dengan menghitung semua kegiatan, tabungan menunjukkan rencana harian, bukan uang yang benar-benar sudah masuk atau keluar. Sisi positifnya, pengguna langsung melihat dampak pengeluaran yang direncanakan dan dapat mengerem lebih awal, serta mendapat gambaran sejak pagi. Sisi negatifnya, pemasukan yang belum diterima (misalnya freelance Rp 50.000 yang belum selesai) sudah dihitung sehingga muncul rasa aman semu: target tampak tercapai padahal uangnya belum ada. Versi "hanya yang selesai" lebih akurat untuk mencatat realisasi, sedangkan versi saat ini lebih cocok sebagai alat perencanaan; idealnya keduanya ditampilkan sebagai rencana versus realisasi.

5. Bagian mana dari aplikasi ini yang paling sulit, dan apa yang akan Anda ubah bila mengerjakannya ulang?

Bagian yang paling menantang adalah menyinkronkan tampilan antar tab dan mengatur tata letak kartu ringkasan. Menjaga tiga tab tetap sinkron membutuhkan pola versiData dan disiplin memasang serta melepas listener. Pada tata letak, kartu ringkasan sempat mengalami overflow 21 piksel karena tiga teks dalam satu Row tanpa pembagian ruang. Bila mengerjakan ulang, saya akan merancang tata letak kartu dan mengujinya pada ukuran layar kecil lebih awal, menyembunyikan kategori "-" pada kegiatan non-pengeluaran, serta menambahkan pengujian otomatis untuk query laporan.

13. Kesimpulan

Mini proyek TabungKu berhasil menyatukan materi Pertemuan 1–6 dalam satu aplikasi utuh: widget dan layout membentuk tampilan, form dan validasi mengatur input kegiatan, SQLite menyimpan data, query agregasi (COUNT, SUM, CASE WHEN, GROUP BY) menghasilkan laporan harian, shared_preferences menyimpan tema, warna, dan target, named routes dan NavigationBar mengatur navigasi, serta animasi implisit menampilkan kemajuan menuju target. Pola ValueNotifier versiData menjaga seluruh tab tetap sinkron, dan struktur folder yang dipisah ke model, data, state, widget, dan halaman membuat kode mudah dirawat. Pengujian menunjukkan aplikasi menghitung pemasukan, pengeluaran, tabungan, serta peringatan dan apresiasi target dengan benar, dengan catatan perbaikan tata letak pada kartu ringkasan.